

1757

A DIGITAL SOCIAL DEDUCTION GAME

Final Presentation

Group 3

Md. Saif-Al Sarker | M Safwan Hasan Khan | Wasi Omar Syed

230042106 | 230042117 | 230042143

What is 1757?

A hidden-role game set at the Battle of Plassey, 1757: two sides fight over Bengal, plus one wildcard.

NAWAB'S LOYALISTS

The Nawab's loyal court. They don't know who the traitors are. Win 3 rounds to save Bengal.

EAST INDIA COMPANY

Traitors hidden inside the court, working with the Company. They sabotage missions. Win 3 rounds to take Bengal.

6–15 players, 5 rounds, first side to win 3. Every player gets a secret role, so the game is won by talking, voting and reading people, live in the browser or on mobile.

KEY FEATURES

Rooms & Access

- Sign up, log in, or play instantly as a guest
- Public or private rooms with a 6-character code
- Host sets the timer, enabled roles and player cap

Real-Time Gameplay

- Text chat plus peer-to-peer voice chat
- Nominate, vote and play mission cards instantly
- 30-second reconnect grace; state saved to Redis

Secret Roles

- 15 historical roles plus 2 filler roles
- Every player sees only what their role allows
- The server checks every move, so no cheating

Profiles & Platforms

- XP, levels, win rate and match history
- Web app, and admin dashboard
- Live online: API on Render, web on Vercel

How a Round Works

Discuss & Nominate

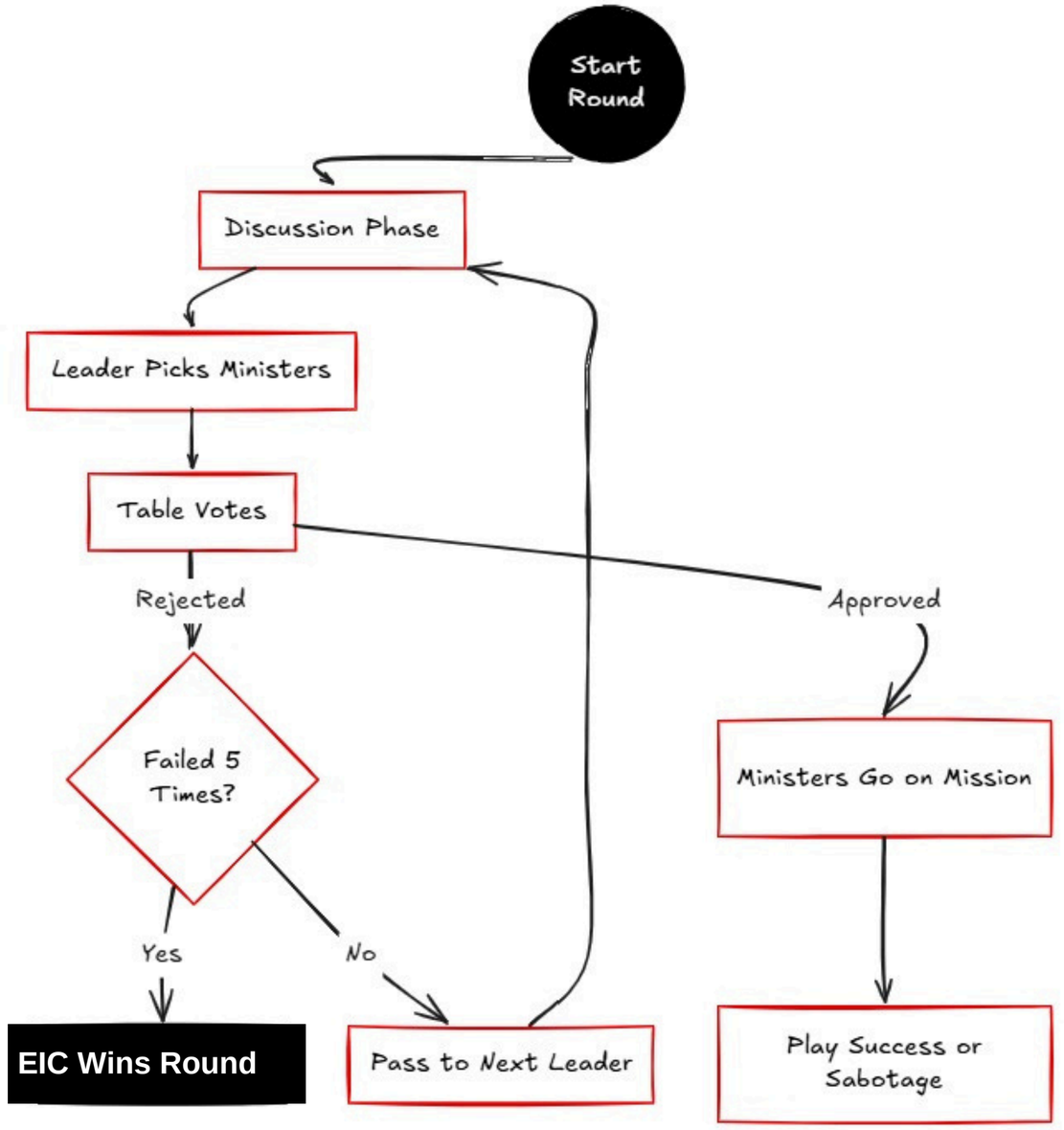
The court debates in chat under a timer (30–300 s, set by the host), then the Leader picks a team of ministers.

Court Ballot

Everyone votes Approve or Reject; ties reject. A rejection passes leadership on and goes back to Discussion 5 rejections in one round give it to EIC.

Mission & Result

Ministers secretly play Success or Sabotage (Greens can only play Success). One Sabotage fails the mission; some missions need 2. First side to 3 rounds goes to the Endgame.



ROLES IN THIS DEMO

NAWAB'S LOYALISTS

Nawab Siraj: always sees Lutfunnesa

Lutfunnesa: always sees the Nawab

Mir Madan: sees the Reds in Round 1 Discussion

Saint Frais: his ballot vote secretly counts twice

EAST INDIA COMPANY

Tara Bai: confuses the lovers, so the Nawab and Lutfunnesa each see two partners

Mir Jafar: if Green wins, he steals the win by naming Mir Madan

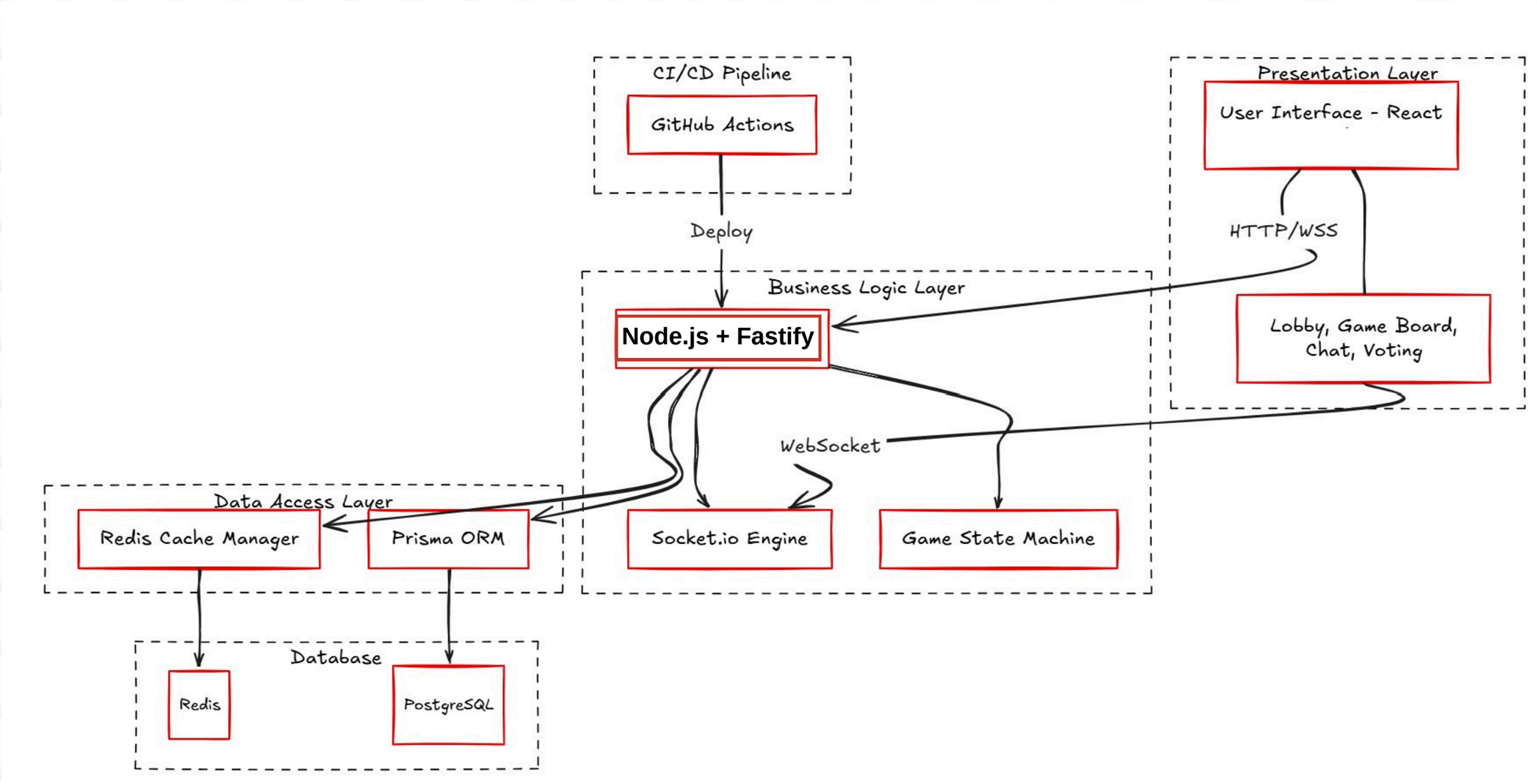
Four kinds of hidden info: a decoy ally, a spy's glimpse, a secret double vote, an endgame steal.

LIVE DEMO

6 players · 4 Green vs 2 Red · 1 full round, live

Watch for: each player's secret role card · the court ballot · a hidden Sabotage · the round result

Software Architecture (3-Tier Model)



Finite State Machine (FSM) Flow

Deterministic Logic

FSM : (state, action, now, rng) → { state, events }

State Pipeline

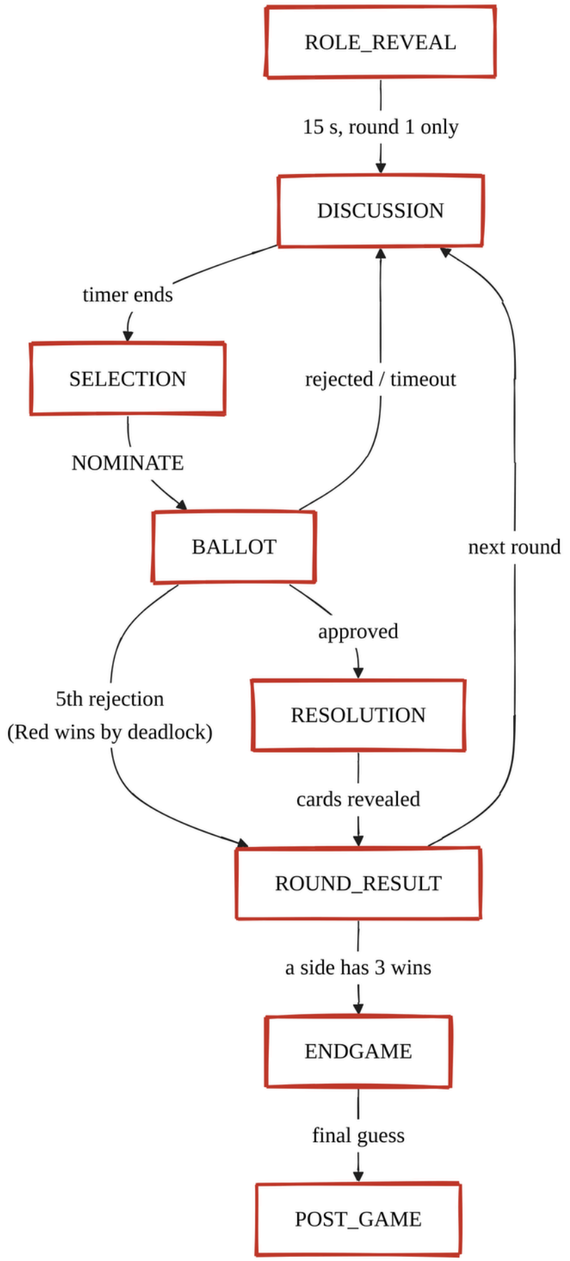
Game Flow : Role Reveal → Discussion → Selection

Decision Loops

Voting : Approved → Resolution; Rejected → Discussion

Endgame Condition

Round Result : 3 wins → Endgame guess → Post-game



TECH STACK

One TypeScript monorepo, managed with pnpm + Turborepo

Frontend

- **Web game:** Next.js 16 (Turbopack), React 19, Tailwind CSS 4 + daisyUI 5
- **Admin panel:** Next.js 16 + Tailwind
- **Browser:** Socket.io client, WebRTC voice chat (P2P mesh, Google STUN), HTML5 Audio

Backend (apps/api)

- Fastify with helmet, CORS and rate limiting
- Socket.io for the real-time game, chat and voice signaling
- Zod input validation
- Redis via ioredis, falls back to memory
- Built with tsup

Shared Packages & Data

- **Game-engine:** the game rules in pure TypeScript, used by both the API and the web app
- **Database:** Neon (serverless PostgreSQL) + Prisma 7 ORM with the Neon adapter
- **Auth:** Better Auth with guest sign-in and bearer tokens

Testing: Vitest · Deploy & tooling: Docker + docker-compose, Render (render.yaml), GitHub Actions CI, Prettier

PROJECT STRUCTURE

Turborepo + pnpm monorepo: 4 apps we deploy, 8 packages they share

apps/ (what we deploy)

```
1757-RPG/
├── apps/
│   ├── api/   Fastify + Socket.IO server
│   │   └── src/
│   │       ├── socket/   event wiring
│   │       ├── realtime/   game, timers, Redis
│   │       ├── lobby/   rooms, codes, rules
│   │       ├── persistence/   match recorder
│   │       └── admin/  users/  routes/  lib/
│   ├── web/   Next.js game
│   │   └── app/  components/  lib/
│   └── admin/   telemetry dashboard
│
```

packages/ (shared code)

```
├── packages/
│   ├── game-engine/   the rules (FSM)
│   │   ├── src/
│   │   │   ├── reduce.ts   createGame.ts
│   │   │   ├── phases/   one file per phase
│   │   │   ├── roles/  abilities/  vision/
│   │   │   └── config/   gameConfig.ts
│   │   └── test/   24 Vitest suites
│   ├── db/   Prisma schema
│   ├── auth/   Better Auth
│   └── types/  config/  ui/  *-config/
├── docs/   .github/workflows/ci.yml
└── Dockerfile   render.yaml   turbo.json
```

Why it's split this way

- apps/ holds what we deploy; packages/ holds the code they share

- Every game rule lives in game-engine. It does no I/O, so the server and the web app run the same rules, and tests need no server

- The API is split by job: socket → realtime → persistence

- The engine is split by phase, role, ability and vision, so a new role touches one folder

- Turborepo rebuilds and retests only what changed

From the git-tracked files. Left out: config boilerplate, build output and the generated Prisma client

COMPONENT COMMUNICATION

Typed Socket.IO events, checked with Zod before they reach the engine

Front end ↔ Back end

- REST: sign-in, rooms, profiles
- Socket.IO, client → server:
  game:action { roomId, action }
- Server → client:
  game:view (private, per player)
  game:event (whole room)
- Every event gets an ack

Back end ↔ Database

- Live game: Redis snapshot after every change
- PostgreSQL via Prisma models:
  User { name, xp, level }
  Match { reason, roundsGreen, roundsRed }
  BallotVote { seat, vote, weight }
- Match history is written once, when the game ends

One request: a ballot vote

1. Click Approve → game:action
   { type: "CAST_VOTE", vote: "APPROVE" }
2. Zod validates; playerId comes from the session
3. reduce() updates the game; saved to Redis
4. Everyone gets a new game:view; the room sees the headcount only
5. Game ends → saved as a BallotVote row

CLASSES & METHODS

Small functions, each with one job

How the code is divided

- Only 7 classes in ~9,550 lines, used where state lives: GameService, snapshot stores, voice rooms, rate limiter, audio
- Everything else is plain functions and React function components
- Engine: one module per FSM phase; roles are data; abilities and vision are separate modules
- API: socket, gameService, scheduler, snapshotStore, roomRepository each do one thing

Why

- Single responsibility: each module has one reason to change
- Pure core, I/O at the edges: only apps/api touches sockets, Redis and Postgres
- Dependencies point one way: apps → packages, never back
- A new role is a catalog entry plus a vision rule, with no edits to the FSM

Size (lines of code)

Methods (363)
avg 22.7 · median 8 · max 930
Classes (7)
avg 111.7 · max 345 (GameService)
Modules (102 files)
avg 93.6 · max 899
Game engine methods
avg 13.0 · max 78

Why 930? It's RoomPage, a React component whose whole JSX markup counts as one function: the one real outlier and the next thing we'd split. GameService (345) is the one class that owns live matches and timers. Measured with the TypeScript compiler; tests and generated code skipped.

TEAM CONTRIBUTION

Wasi Omar Syed
ID **230042143**
- Game-engine state machine: phases, timers, transitions
- Rounds, court ballot & mission resolution
- Role assignment & role powers
- Rooms: create and join by ID

Md. Saif-Al Sarker
ID **230042106**
- Auth with Better Auth: login, sign-up & guests
- Real-time chat over Socket.IO
- Sound Effects
- Admin dashboard
- Automated tests (Vitest)

M Safwan Hasan Khan
ID **230042117**
- Docker & deployment: Render API, Vercel web/admin
- Data layer: Redis live state, Prisma + PostgreSQL
- CI/CD pipeline
- Web UI: lobby, game table & profiles

All three of us shared the design work, testing and documentation.

THANK YOU

Questions?